LESSON 2: VARIABLES & DATA TYPES IN PYTHON

Python Programming Foundation Series — In-Depth Study Notes & Architecture Reference

1. VARIABLES: THE MEMORY REFERENCE MODEL

In low-level compiled languages like C or C++, a variable is a named storage location—a physical "box" in memory with a fixed data type and fixed byte size (e.g., int x = 10;).

In Python, **variables are not containers; they are labels or reference pointers to objects stored in the memory heap**.

1.1 The Reference Model Explained

When you execute the statement:

Python

```
x = 10
```

Python performs three distinct steps:

1. **Object Allocation:** Allocates an integer object representing value 10 in heap memory.
2. **Type Tagging:** Tags that object with metadata indicating its class (int) and an internal reference count.
3. **Reference Binding:** Binds the identifier x as a pointer to the memory address of that 10 object.

Variable Identifier (Name Tag) ——▶ Heap Memory Object

[x] ——▶ [Address: 0x10A | Type: int | Value: 10]

If you reassign x:

Python

```
x = "Python"
```

The variable label x is simply moved to point to a new string object in heap memory. The original integer object 10 loses that reference, and if no other variables reference it, it is automatically deallocated by Python's garbage collector.

2. DYNAMIC TYPING VS STRONG TYPING

Python belongs to a specific linguistic classification that is frequently misunderstood: it is **dynamically typed**, yet **strongly typed**.

Dynamic Typing

Variable names carry no fixed data type. The type lives on the object in memory, not on the variable name itself.

Python

```
data = 42        # data points to an int object
data = "Universal" # data now points to a str object (Completely valid)
```

Strong Typing

Python enforces strict type boundaries. Objects of incompatible types will not implicitly coerce into another type during operations:

Python

```
# ❌ RAISES TypeError: Python will NOT auto-convert int to string
total = "Total score: " + 100


# ✅ CORRECT: Explicit conversion required
total = "Total score: " + str(100)
```

3. VARIABLE IDENTIFIER RULES & PEP 8 STANDARDS

To write maintainable code that complies with Python's compiler and official style guides (**PEP 8**):

Lexical Rules (Enforced by Interpreter)

- Must begin with an uppercase/lowercase letter (a-z, A-Z) or an underscore (_).
- May contain letters, digits (0-9), and underscores (_).
- Cannot begin with a number (e.g., 2nd_user is invalid; user_2nd is valid).
- Case-sensitive (Age, age, and AGE are three distinct variables).
- Cannot use Python reserved keywords (if, def, class, import, while, True, None, etc.).

PEP 8 Naming Conventions

- **Variables & Functions:** snake_case (e.g., student_age, total_amount).
- **Constants:** UPPER_SNAKE_CASE (e.g., MAX_RETRIES, PI_VALUE).

- **Classes:** PascalCase (e.g., StudentRecord, DatabaseConnection).

4. BUILT-IN DATA TYPES TAXONOMY

Category	Type	Mutability	Definition & Typical Range	Example
Numeric	int	Immutable	Whole numbers of arbitrary precision (no overflow)	42, -10, 1_000_000
Numeric	float	Immutable	Double-precision IEEE 754 floating-point decimals	3.14159, -0.001, 1e-4
Numeric	complex	Immutable	Numbers with real and imaginary parts ($a + bj$)	3 + 4j
Text	str	Immutable	Unicode character sequence	"Hello", 'Python'
Boolean	bool	Immutable	Logical truth values (True or False), subclass of int	True, False
Sequence	list	**Mutable**	Ordered, heterogeneous collection of objects	[1, "two", 3.0]
Sequence	tuple	Immutable	Ordered, heterogeneous collection of objects	(1, "two", 3.0)
Mapping	dict	**Mutable**	Key-value hash map; unique hashable keys	{"id": 101, "role": "admin"}
Set	set	**Mutable**	Unordered collection of unique hashable elements	{1, 2, 3, 4}

Category	Type	Mutability	Definition & Typical Range	Example
Null	NoneType	Immutable	Represents the absence of a value or null state	None

5. IN-DEPTH NUMERIC TYPES & OPERATORS

5.1 Arbitrary Precision Integers (int)

In languages like C or Java, integers are constrained by hardware registers (e.g., 32-bit: -2^{31} to $2^{31}-1$). In Python 3, integers have **arbitrary precision**—they can expand to fill available RAM without integer overflow errors:

Python

```
massive_number = 10**100  # Googol: 1 followed by 100 zeroes (Executes effortlessly)
```

5.2 Floating-Point Nuances (float)

Floats are implemented as C double primitives (64-bit IEEE 754). Because computers represent fractions in binary (base-2), some base-10 decimals cannot be represented with absolute precision:

Python

```
0.1 + 0.2 == 0.3  # Returns False! (Internally evaluates to 0.30000000000000004)
```

Note: For financial calculations requiring exact precision, use Python's built-in decimal.Decimal module.

5.3 Division Operators: True vs. Floor

- **True Division (/):** Always returns a float, even if the division divides evenly: 10 / 2 yields 5.0.
- **Floor Division (//):** Divides and rounds down to the nearest mathematical integer: 10 // 3 yields 3, and -10 // 3 yields -4.

6. STRINGS (str): IMMUTABILITY, SLICING & FORMATTING

A string is an **immutable sequence of Unicode code points**. Once instantiated in memory, individual characters cannot be mutated in place.

Python

name = "Python"

name[0] = "J" # ❌ TypeError: 'str' object does not support item assignment

6.1 Indexing & Slicing Syntax

Python sequences support zero-based positive indexing and negative (reverse) indexing.

P y t h o n

0 1 2 3 4 5 (Forward index)

-6 -5 -4 -3 -2 -1 (Negative index)

Slice Formula: sequence[start:stop:step] *(Includes start, excludes stop)*

- text[0:3] → "Pyt" (Indices 0, 1, 2)
- text[2:] → "thon" (From index 2 to end)
- text[::-1] → "nohtyP" (Reverses the string using a step of -1)

6.2 String Formatting Paradigms

Python

user = "Rahul"

score = 98.5

1. f-strings (Modern standard, Python 3.6+):

formatted = f"Candidate {user} attained {score:.1f}%"

2. str.format() method:

formatted = "Candidate {} attained {:.1f}%".format(user, score)

3. %-formatting (Legacy printf style):

formatted = "Candidate %s attained %.1f%%" % (user, score)

7. BOOLEANS & THE CONCEPT OF "TRUTHY" VS "FALSY"

The bool data type contains only two singleton instances: True and False.

Because bool inherits from int:

Python

```
isinstance(True, int)  # True

True + True          # Evaluates to 2
```

Truth Value Testing (Truthy & Falsy)

In Python, every object can be tested for a truth value in an if or while condition.

The following values are inherently FALSY:

- Constants: None, False
- Zero across numeric types: 0, 0.0, 0j
- Empty collections and sequences: "" (empty string), [] (empty list), () (empty tuple), {} (empty dict), set()
- Custom objects implementing a __bool__() or __len__() method that returns False or 0.

All other values, objects, and populated collections evaluate to TRUTHY.

8. TYPE INSPECTION & TYPE CONVERSION (CASTING)

8.1 Inspection: type() vs isinstance()

- type(obj) returns the exact class of the object.
- isinstance(obj, class_info) checks if an object is an instance of a specified class **or any derived subclass**. It also accepts a tuple of types.

Python

```
class Animal: pass

class Dog(Animal): pass


d = Dog()

type(d) == Animal       # False (Strict check)

isinstance(d, Animal)    # True  (Respects inheritance; preferred in production)
```

8.2 Explicit Type Conversion

Python

```
# 1. Truncating conversion (float -> int drops decimals, does NOT round)
int(9.99)      # 9

# 2. String parsing
int("150")     # 150
float("3.1415") # 3.1415

# 3. Parsing failures
# int("3.14")   # ❌ ValueError: invalid literal for int() with base 10: '3.14'
int(float("3.14")) # ✅ 3 (Two-step conversion)
```

9. COMPLETE PRODUCTION CODE SCRIPT

Below is a self-contained, end-to-end Python file implementing variables, dynamic typing, numeric systems, string slicing, truthiness evaluation, and data validation:

```
Python
"""
Lesson 2: Variables and Built-in Data Types
A comprehensive demonstration of memory references, operations,
type casting, truthiness, and defensive data validation.
"""

# ==============================================================================
# 1. MULTIPLE ASSIGNMENT & REFERENCE BINDING
# ==============================================================================
print("--- 1. Variable Assignment & References ---")

# Multiple variables assigned different objects simultaneously
```

```
student_id, student_name, current_gpa = 101, "Rahul Sharma", 8.75
print(f"ID: {student_id} | Name: {student_name} | GPA: {current_gpa}")

# Multiple variables bound to the SAME memory object
val_a = val_b = val_c = 500
print(f"Values: {val_a}, {val_b}, {val_c}")
print(f"Memory Identifiers: id(val_a)={id(val_a)} == id(val_b)={id(val_b)}")

# Dynamic typing: Changing identifier's target type
state_tracker = "Initializing"
print(f"state_tracker: '{state_tracker}' [{type(state_tracker).__name__}]")
state_tracker = 200
print(f"state_tracker: {state_tracker} [{type(state_tracker).__name__}]")

# ========================================================================
# 2. NUMERIC DATA TYPES & MATHEMATICAL OPERATIONS
# ========================================================================
print("\n--- 2. Numbers Engine ---")

integer_val = 42
float_val = 3.14159
complex_val = 4 + 5j

print(f"Complex Number: {complex_val}")
print(f"Real Part: {complex_val.real} | Imaginary Part: {complex_val.imag}")
```

```
# Floor division vs True division
print(f"True Division (17 / 5):  {17 / 5}")   # Returns 3.4 (float)
print(f"Floor Division (17 // 5): {17 // 5}") # Returns 3 (int)
print(f"Modulus (17 % 5):       {17 % 5}")   # Returns 2 (remainder)
print(f"Exponentiation (2 ** 8):  {2 ** 8}")   # Returns 256

# ============================================================================
# 3. STRINGS: SLICING, IMMUTABILITY & METHODS
# ============================================================================
print("\n--- 3. String Processing & Slicing ---")

raw_message = "   enterprise-python-developer   "
clean_message = raw_message.strip()

print(f"Raw Input: '{raw_message}'")
print(f"Stripped:  '{clean_message}' (Length: {len(clean_message)})")
print(f"Uppercase: '{clean_message.upper()}'")
print(f"Replaced:  '{clean_message.replace('python', 'backend')}'")

# Indexing & Slicing
sample = "DEVELOPMENT"
print(f"Initial 4 characters [0:4]: {sample[0:4]}")
print(f"Final 4 characters [-4:]:   {sample[-4:]}")
print(f"Step by 2 characters [::2]: {sample[::2]}")
print(f"Reversed String [::-1]:     {sample[::-1]}")
```

```
# ============================================================================
# 4. BOOLEANS & TRUTH VALUE EVALUATION
# ============================================================================
print("\n--- 4. Truthiness & Boolean Expressions ---")

# Testing Falsy structures
falsy_candidates = [0, 0.0, "", [], (), {}, set(), None, False]
for item in falsy_candidates:
    print(f"Value: {str(item):<10} -> bool(): {bool(item)}")

# Truthy check in idiomatic Python
incoming_payload = ["Data Record 1", "Data Record 2"]
if incoming_payload:
    print(f"Payload received ({len(incoming_payload)} items). Processing...")
else:
    print("Empty payload received.")

# ============================================================================
# 5. DEFENSIVE PROGRAMMING VIA TYPE VALIDATION
# ============================================================================
print("\n--- 5. Practical Application: Input Validation Record ---")

def process_student_record(record: dict):
    """
    Validates record attributes defensive-first before processing logic.
    """
```

```
    # Verify name is string
    if not isinstance(record.get("name"), str):
        raise TypeError("Invalid attribute: 'name' must be of type str.")

    # Verify age is an integer
    if not isinstance(record.get("age"), int):
        raise TypeError("Invalid attribute: 'age' must be of type int.")

    # Verify GPA is numeric (float or int)
    if not isinstance(record.get("gpa"), (float, int)):
        raise TypeError("Invalid attribute: 'gpa' must be a numeric type.")

    # Calculate derived age in months
    age_in_months = record["age"] * 12
    return f"Student {record['name'].title()} validated. Age in months: {age_in_months}"

sample_student = {
    "name": "priya patel",
    "age": 22,
    "gpa": 9.4,
    "scholarship": True
}

output = process_student_record(sample_student)
print(output)
```

10. COMMON JUNIOR DEVELOPER ANTI-PATTERNS

Anti-Pattern 1: Comparing Types Directly with == Instead of isinstance()

Python

```
# ❌ ANTI-PATTERN: Breaks polymorphism and class inheritance
if type(user_input) == int:
    pass


# ✅ CORRECT: Accommodates subclasses and tuples of types
if isinstance(user_input, int):
    pass
```

Anti-Pattern 2: Unnecessary Explicit Truthiness Comparison

Python

```
# ❌ VERBOSE & UN-PYTHONIC
if is_enrolled == True:
    pass


if len(student_list) > 0:
    pass


# ✅ IDIOMATIC PYTHON (Leveraging Truthy/Falsy nature)
if is_enrolled:
    pass


if student_list:
    pass
```

Anti-Pattern 3: Inefficient String Concatenation in Loops

Python

```
# ❌ ANTI-PATTERN: Creates O(N^2) memory re-allocations (strings are immutable!)
res = ""
for word in ["Learn", "Python", "Deeply"]:
    res += word + " "

# ✅ OPTIMAL: Single O(N) allocation using join
res = " ".join(["Learn", "Python", "Deeply"])
```

11. TECHNICAL INTERVIEW & RECAP CHEAT SHEET

High-Yield Questions & Answers

Q1: In Python, are variables passed by value or passed by reference?

Answer: Python uses an evaluation strategy called **"Pass-by-assignment"** (or *Pass-by-object-reference*). If you pass an immutable object (e.g., int, str, tuple) to a function, reassigning it inside the function does not affect the caller. If you pass a mutable object (e.g., list, dict), modifying the object in-place (such as list.append()) mutates the original object seen by the caller.

Q2: What is the internal difference between == and is?

Answer: The == operator evaluates **value equality** (checks if the data inside two objects is equivalent). The is operator evaluates **identity equality** (checks if both variable pointers refer to the exact same physical memory address: id(a) == id(b)).

Q3: What are Python's small integer caching and string interning mechanisms?

Answer: In CPython, small integers ranging from -5 **to** 256 are pre-allocated at startup as global singletons. Any variable assigned an integer in this range points to the same pre-existing memory address. Short strings without spaces are similarly interned to optimize memory usage and comparison speed.

Q4: Why does float(0.1) + float(0.2) not equal 0.3 exactly?

Answer: Computers represent floating-point numbers in binary using the IEEE 754 floating-point standard. Fractions whose denominators cannot be expressed as powers of 2 (such as $1/10$ and $2/10$) result in infinitely repeating binary fractions that must be truncated, introducing a minute precision error (0.30000000000000004).